

GUÍA PRÁCTICA #2

NOTA: Revisar y completar la Guía Práctica #1

OPERACIONES: Crea y construye un proyecto en Laravel de una API RESTful.

PASO #1: Implementar el controlador ProductController

El controlador gestionará las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) de productos. En el archivo **app/Http/Controllers/Api/ProductController.php**, agrega lo siguiente:

```
<?php

namespace App\Http\Controllers\Api;

use App\Http\Controllers\Controller;
use App\Models\Product;
use App\Http\Requests\StoreProductRequest;
use App\Http\Requests\UpdateProductRequest;
use App\Http\Resources\ProductResource;
use Illuminate\Http\Request;

class ProductController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        $products = Product::all();
        return ProductResource::collection($products);
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        $product = Product::create($request->validated());
        return new ProductResource($product);
    }
}
```



```
/**
 * Display the specified resource.
 */
public function show(string $id)
{
    $product = Product::findOrFail($id);
    return new ProductResource($product);
}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, string $id)
{
    $product = Product::findOrFail($id);
    $product->update($request->validated());
    return new ProductResource($product);
}

/**
 * Remove the specified resource from storage.
 */
public function destroy(string $id)
{
    $product = Product::findOrFail($id);
    $product->delete();
    return response()->json(null, 204);
}
}
```

PASO #2: Crear las peticiones de validación.

StoreProductRequest (app/Http/Requests/StoreProductRequest.php):

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreProductRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
```



```
{
    return false;
}

/**
 * Get the validation rules that apply to the request.
 *
 * @return array<string,
\Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>
 */
public function rules(): array
{
    return [
        'name' => 'required|string|max:255',
        'description' => 'nullable|string',
        'sku' => 'required|string|max:255',
        'stock' => 'required|numeric',
        'price' => 'required|integer',
        'is_active' => 'nullable|boolean',
    ];
}
}
```

PASO #3: UpdateProductRequest (app/Http/Requests/UpdateProductRequest.php):

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class UpdateProductRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return false;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string,
\Illuminate\Contracts\Validation\ValidationRule|array<mixed>|string>

```



```
*/  
public function rules(): array  
{  
    return [  
        'name' => 'required|string|max:255',  
        'description' => 'nullable|string',  
        'sku' => 'required|string|max:255',  
        'stock' => 'required|numeric',  
        'price' => 'required|integer',  
        'is_active' => 'nullable|boolean',  
    ];  
}  
}
```

PASO #4: Crear el recurso ProductResource

En el archivo `app/Http/Resources/ProductResource.php`, agrega lo siguiente para transformar la respuesta de la API:

```
<?php  
  
namespace App\Http\Resources;  
  
use Illuminate\Http\Request;  
use Illuminate\Http\Resources\Json\JsonResource;  
  
class ProductResource extends JsonResource  
{  
    /**  
     * Transform the resource into an array.  
     *  
     * @return array<string, mixed>  
     */  
    public function toArray(Request $request)  
    {  
        /*return parent::toArray($request);*/  
        return [  
            'id' => $this->id,  
            'name' => $this->name,  
            'description' => $this->description,  
            'sku' => $this->sku,  
            'stock' => $this->stock,  
            'price' => $this->price,  
            'is_active' => $this->is_active,  
            'created_at' => $this->created_at,  
            'updated_at' => $this->updated_at,  
        ];  
    }  
}
```



```
    ];  
  }  
}
```

PASO #5: Rutas de la API. Agrega las rutas correspondientes para los productos en el archivo routes/api.php:

```
<?php  
  
use App\Http\Controllers\Api\ProductController;  
  
Route::apiResource('products', ProductController::class);
```

NOTA (Error #1): Si no tienes el archivo “api.php” dentro de la carpeta routes, ejecuta el siguiente comando:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/tienda-api (main)  
● $ php artisan install:api
```

¿Qué hace exactamente este comando?

- **Crea el archivo:** Generará el archivo routes/api.php con la estructura base correcta.
- **Registra el prefijo:** Le dirá a Laravel que todas las rutas escritas en ese archivo deben llevar automáticamente el prefijo /api en la URL (ej. [tu-dominio.com/api/productos]).

Este comando generará las rutas para las operaciones CRUD de los productos (GET /products, POST /products, PUT /products/{id}, DELETE /products/{id}).



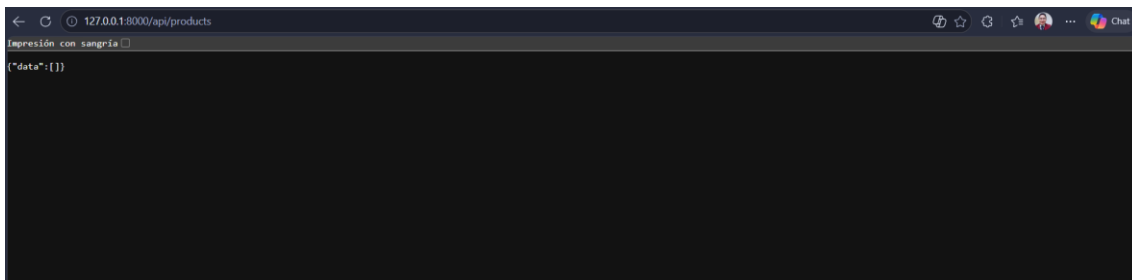
PASO #6: Probar API. Una vez que hayas configurado todo, puedes ejecutar el servidor de desarrollo de Laravel:

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/actividad-01
$ php artisan serve
```

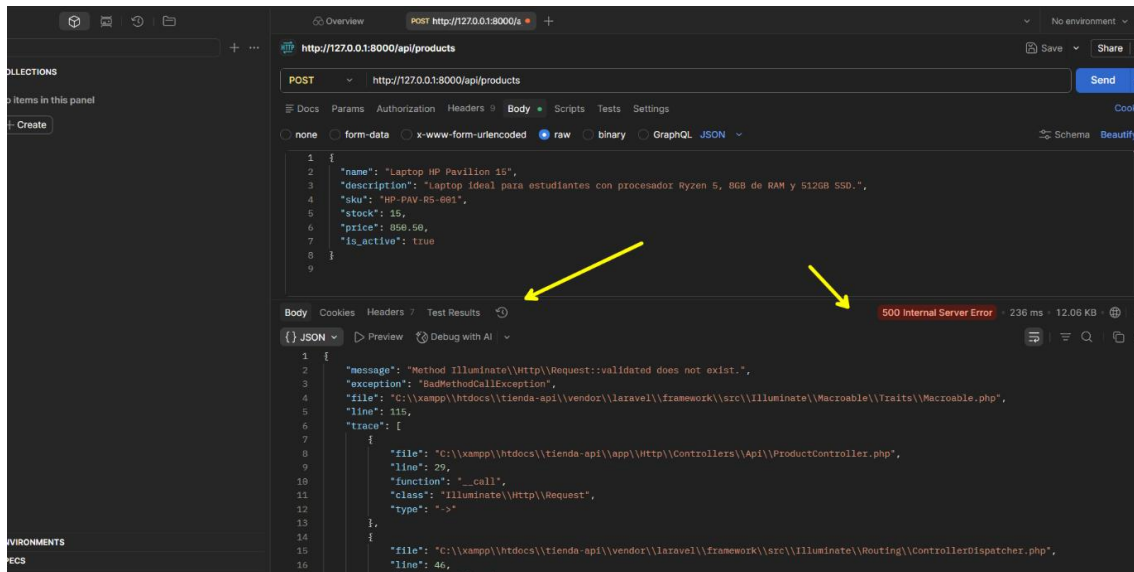
La API debería estar accesible en <http://127.0.0.1:8000/api/products>. Puedes probar las operaciones de creación, consulta, actualización y eliminación utilizando herramientas como [Postman](#).

Resumen de operaciones

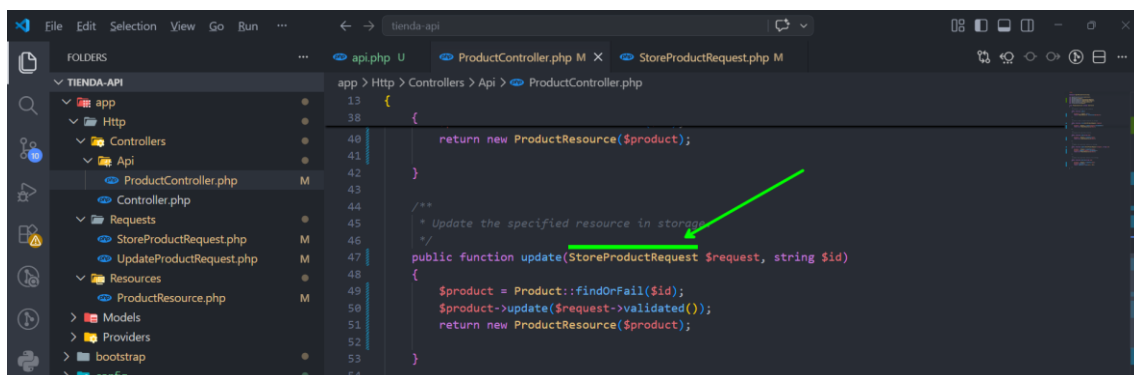
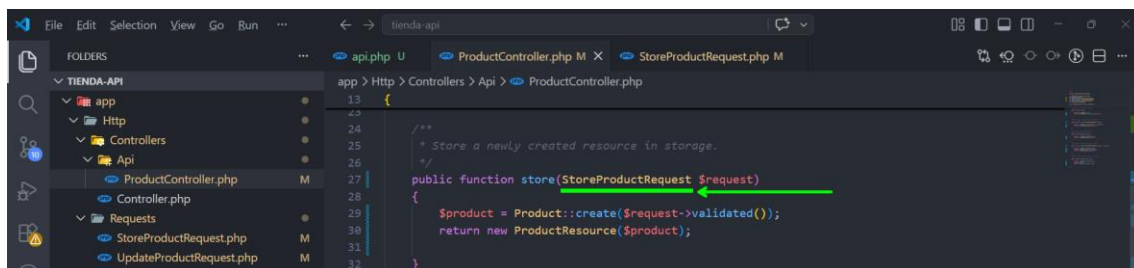
- **GET /api/products:** Lista todos los productos.
- **GET /api/products/{id}:** Muestra un producto por ID.
- **POST /api/products:** Crea un nuevo producto.
- **PUT /api/products/{id}:** Actualiza un producto.
- **DELETE /api/products/{id}:** Elimina un producto.

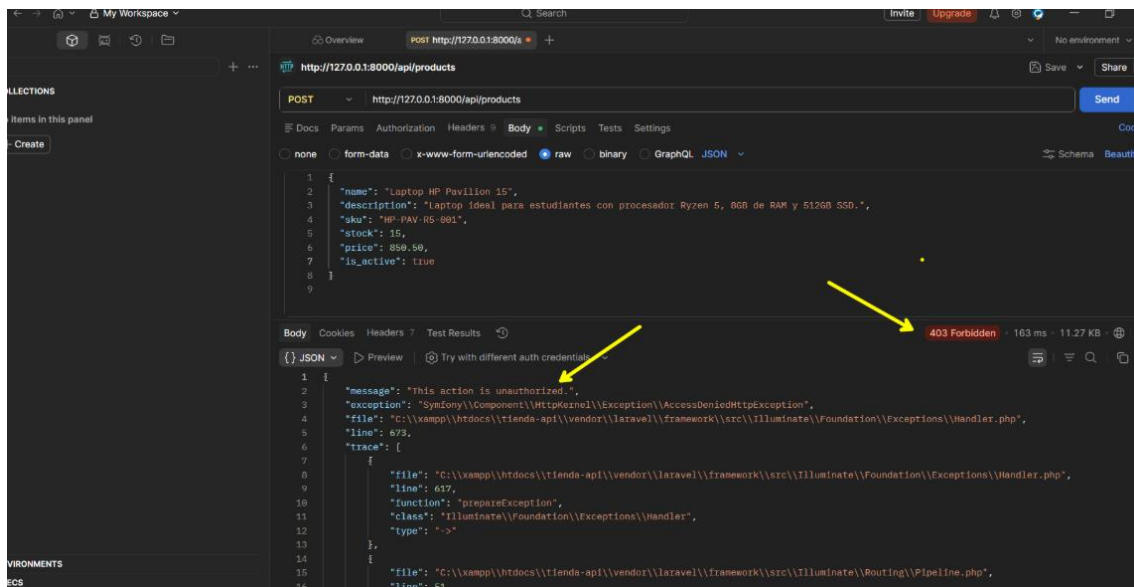


PASO #7: ¿Sale error 500 Internal Server Error?

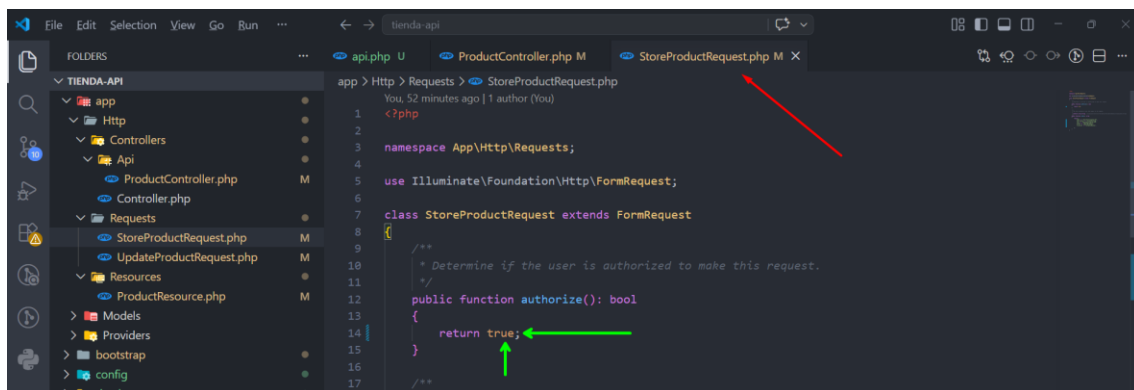
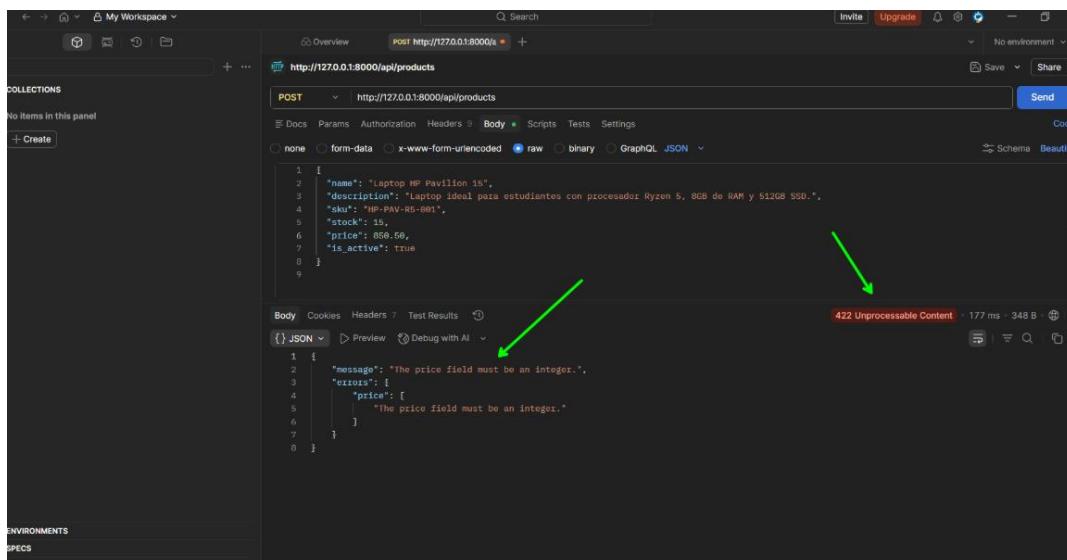


Abre el archivo `ProductController.php` y corrige lo siguiente:

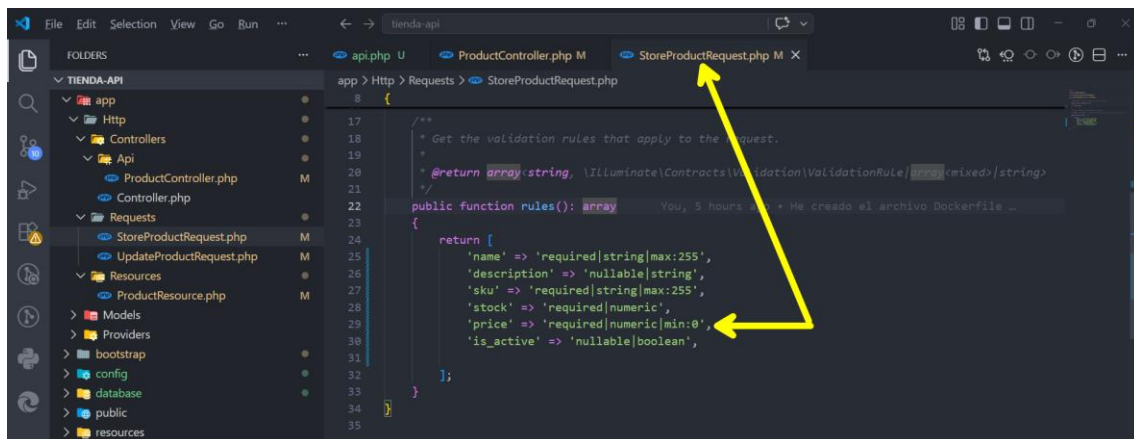


PASO #8: ¿Sale error 403 Forbidden?

Abre el archivo `StoreProductRequest.php` y corrige lo siguiente:

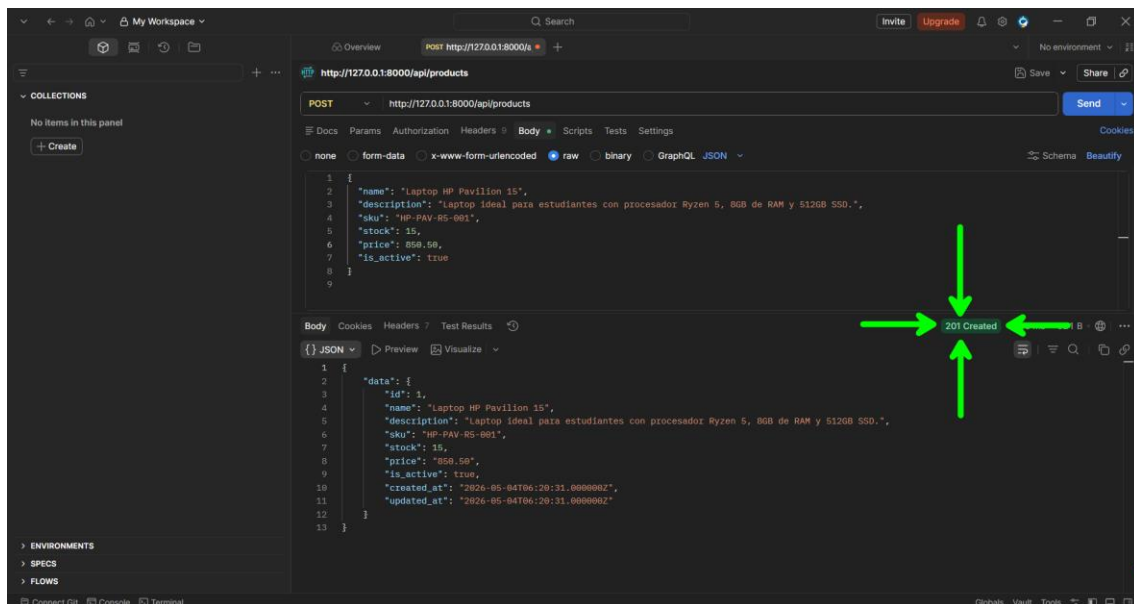
**PASO #9: ¿Sale error 422 Unprocessable Content?**

Abre el archivo StoreProductRequest.php y corrige lo siguiente:



```
8 {
17 /**
18  * Get the validation rules that apply to the request.
19  *
20  * @return array, string, \Illuminate\Contracts\Validation\ValidationRule|array|string (mixed)/string
21  */
22 public function rules(): array {
23     // You, 5 hours ago · He creado el archivo Dockerfile ...
24     return [
25         'name' => 'required|string|max:255',
26         'description' => 'nullable|string',
27         'sku' => 'required|string|max:255',
28         'stock' => 'required|numeric',
29         'price' => 'required|numeric|min:0',
30         'is_active' => 'nullable|boolean',
31     ];
32 }
33
34
35 }
```

Guarda tu archivo StoreProductRequest.php, vuelve a Postman y dale a **Send**. ¡Ahora sí, EUREKA, ¡prepárate para ver ese anhelado Código 201 en verde!



```
1 {
2   "name": "Laptop HP Pavilion 15",
3   "description": "Laptop ideal para estudiantes con procesador Ryzen 5, 8GB de RAM y 512GB SSD.",
4   "sku": "HP-PAV-R5-8GB",
5   "stock": 15,
6   "price": 850.50,
7   "is_active": true
8 }
9
```

```
1 {
2   "data": {
3     "id": 1,
4     "name": "Laptop HP Pavilion 15",
5     "description": "Laptop ideal para estudiantes con procesador Ryzen 5, 8GB de RAM y 512GB SSD.",
6     "sku": "HP-PAV-R5-8GB",
7     "stock": 15,
8     "price": "850.50",
9     "is_active": true,
10    "created_at": "2024-05-04T06:20:31.000000Z",
11    "updated_at": "2024-05-04T06:20:31.000000Z"
12  }
13 }
```